```
import numpy as np
```

```
def sigmoid(x): #defining sigmoid activation function
    return 1/(1+np.exp(-x))
```

```
x=np.array([[1,0,0], #input data (with bias term in first column)
            [1,0,1],
            [1,1,0],
            [1,1,1]])
```

```
#expected output (same output as XOR gate)
t=np.array([[0],
           [1],
           [1],
            [0]])
```

```
#initiaze random weights
np.random.seed(42) #for reproducibility
w1=np.random.randn(3,4)
print("w1:",w1)#weights of no. of input units->no. of hidden units
w2=np.random.randn(4,1)
print("\nw2:",w2)#weights of no. of hidden units->no. of output units
```

```
w1: [[ 0.49671415 -0.1382643   0.64768854  1.52302986]
 [-0.23415337 -0.23413696  1.57921282  0.76743473]
 [-0.46947439  0.54256004 -0.46341769 -0.46572975]]

w2: [[ 0.24196227]
 [-1.91328024]
 [-1.72491783]
 [-0.56228753]]
```

```
#----forward pass----
z1=np.dot(x,w1)
print("hin:\n\n",z1) #input layer
a1=sigmoid(z1)
print("\nhout:\n\n",a1)
z2=np.dot(a1,w2)
print("\nYin:\n\n",z2)#yin
y_pred=sigmoid(z2)
print("\nY_pred before error calc")
print("\n",y_pred)
```

```
hin:

 [[ 0.49671415 -0.1382643   0.64768854  1.52302986]
 [ 0.02723977  0.40429574  0.18427085  1.0573001 ]
 [ 0.26256078 -0.37240126  2.22690135  2.29046459]
 [-0.20691361  0.17015879  1.76348366  1.82473483]]

hout:

 [[0.62168683 0.46548889 0.65648939 0.82098421]
 [0.50680952 0.59971932 0.5459378  0.74217425]
 [0.56526568 0.40796092 0.90263938 0.90808424]
 [0.44845537 0.54243735 0.85364543 0.8611333 ]]

Yin:

 [[-2.33420537]
 [-2.38381551]
 [-2.71135381]
 [-2.88599813]]

Y_pred before error calc

 [[0.08832943]
 [0.0844152 ]
 [0.06230671]
 [0.05285008]]
```

```
#learning rate
n=0.1
#no. of iterations of training (epochs)
epochs=10000
```

```
for epoch in range(epochs):
    #----Forward pass----
    net_hidden=np.dot(x,w1) #net input to hidden layer
    o_hidden=sigmoid(net_hidden) #output of hidden layer
    net_output=np.dot(o_hidden,w2) #net input to output layer
    o_output=sigmoid(net_output) #final output (y_pred)
print(o_output)
```

```
[[0.08832943]
 [0.0844152 ]
 [0.06230671]
 [0.05285008]]
```

```
#learning rate
n=0.1
#no. of iterations of training (epochs)
epochs=100001
for epoch in range(epochs):
    #----Forward pass----
    net_hidden=np.dot(x,w1) #net input to hidden layer
    o_hidden=sigmoid(net_hidden) #output of hidden layer
    net_output=np.dot(o_hidden,w2) #net input to output layer
    o_output=sigmoid(net_output) #final output (y_pred)
    #----Backward pass----
    err_output=o_output*(1-o_output)*(t-o_output)
    err_hidden=o_hidden*(1-o_hidden)*np.dot(err_output,w2.T)
    w2+=n*np.dot(o_hidden.T,err_output) #▲wij for w2
    w1+=n*np.dot(x.T,err_hidden) #▲wij for w1
#print error occasionally
    if epoch %20000==0:
        E=0.5*np.sum((t-o_output)**2) #Mean square error
        print(f"Epoch {epoch}, Error: {E:f}")
```

```
Epoch 0, Error: 0.864080
Epoch 20000, Error: 0.001556
Epoch 40000, Error: 0.000690
Epoch 60000, Error: 0.000440
Epoch 80000, Error: 0.000322
Epoch 100000, Error: 0.000254
```

```
print("\n\nOut put before roundng off:\n\n",o_output)
print("\n\nNow output:\n\n",np.round(o_output))
print("\n\n",t==np.round(o_output))
```

```
Out put before roundng off:

 [[0.00581046]
 [0.98730824]
 [0.98916896]
 [0.01398368]]


Now output:

 [[0.]
 [1.]
 [1.]
 [0.]]


 [[ True]
 [ True]
 [ True]
 [ True]]
```

Start coding or generate with AI.

Start coding or generate with AI.